

Week 2 - Day 9a: Image Filters

You will be working on a series of exercises for the material covered in lecture, including:

- **Inverting Colors**
- **(Half) Grayscale Conversion**
- **Flip** (Horizontal and Vertical)
- **Rotation** (Left and Right)

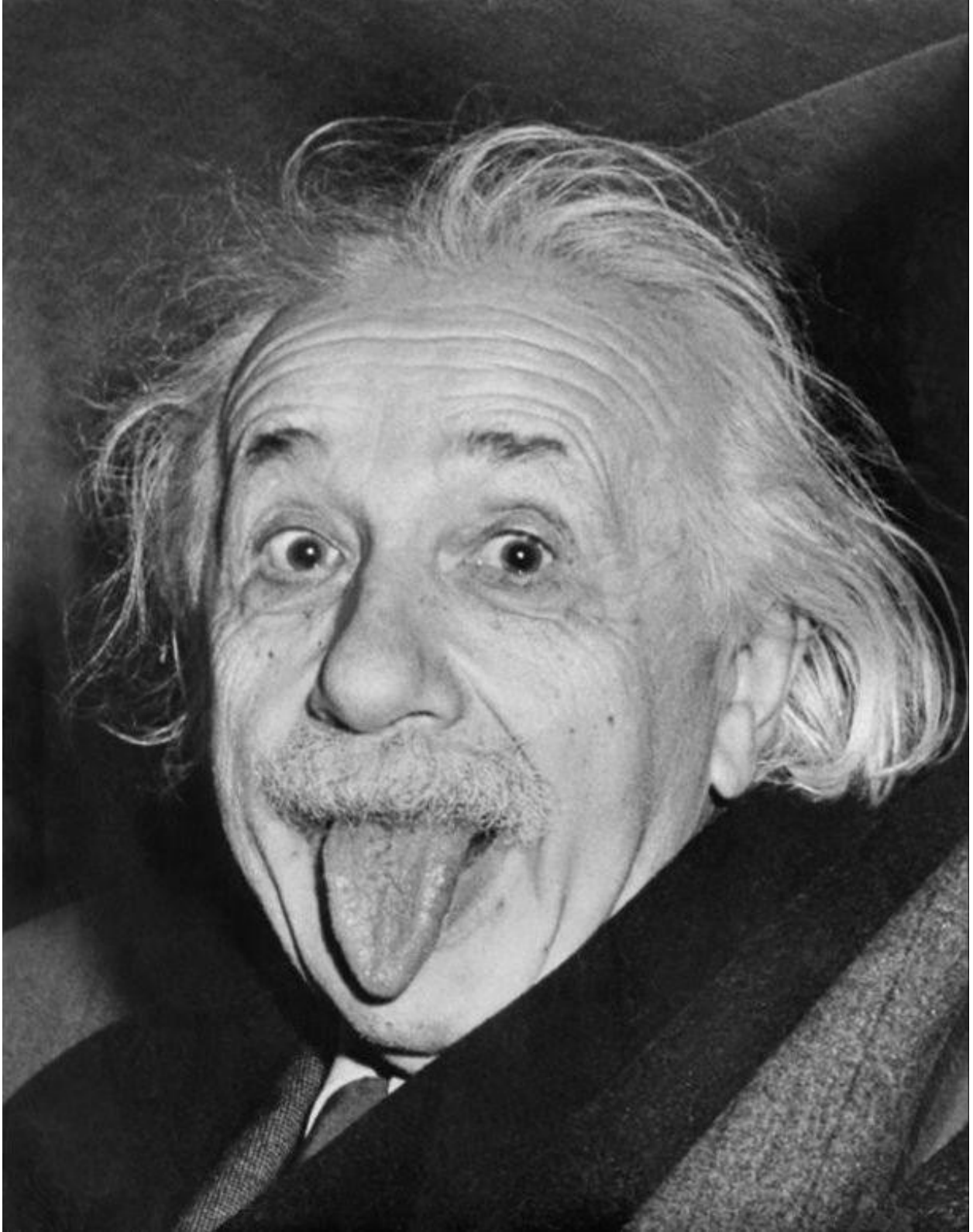
Before doing anything, let's import our simple image class again. This class will be used to load and display images throughout the notebook.

```
from simpleimage import SimpleImage
```

Exercise 1: Inverting Images []

We've seen in class how we can use filters to transform real and interesting images. In this exercise we are going to write more of our own fun filters. To start, let's load a picture of Einstein. We can load it by passing the path to the image file to the `SimpleImage` class.

```
# Load and show `assets/einstein.jpeg`  
path = "assets/einstein.jpeg"  
einstein = SimpleImage(path)  
einstein.show()
```



Let's **invert** the colors on this image. Inverting means that we will take the color of each pixel and subtract it from the maximum value of the color channel. This will make red pixels not red, but cyan, green pixels magenta, and blue pixels yellow. A single pixel can be represented as a

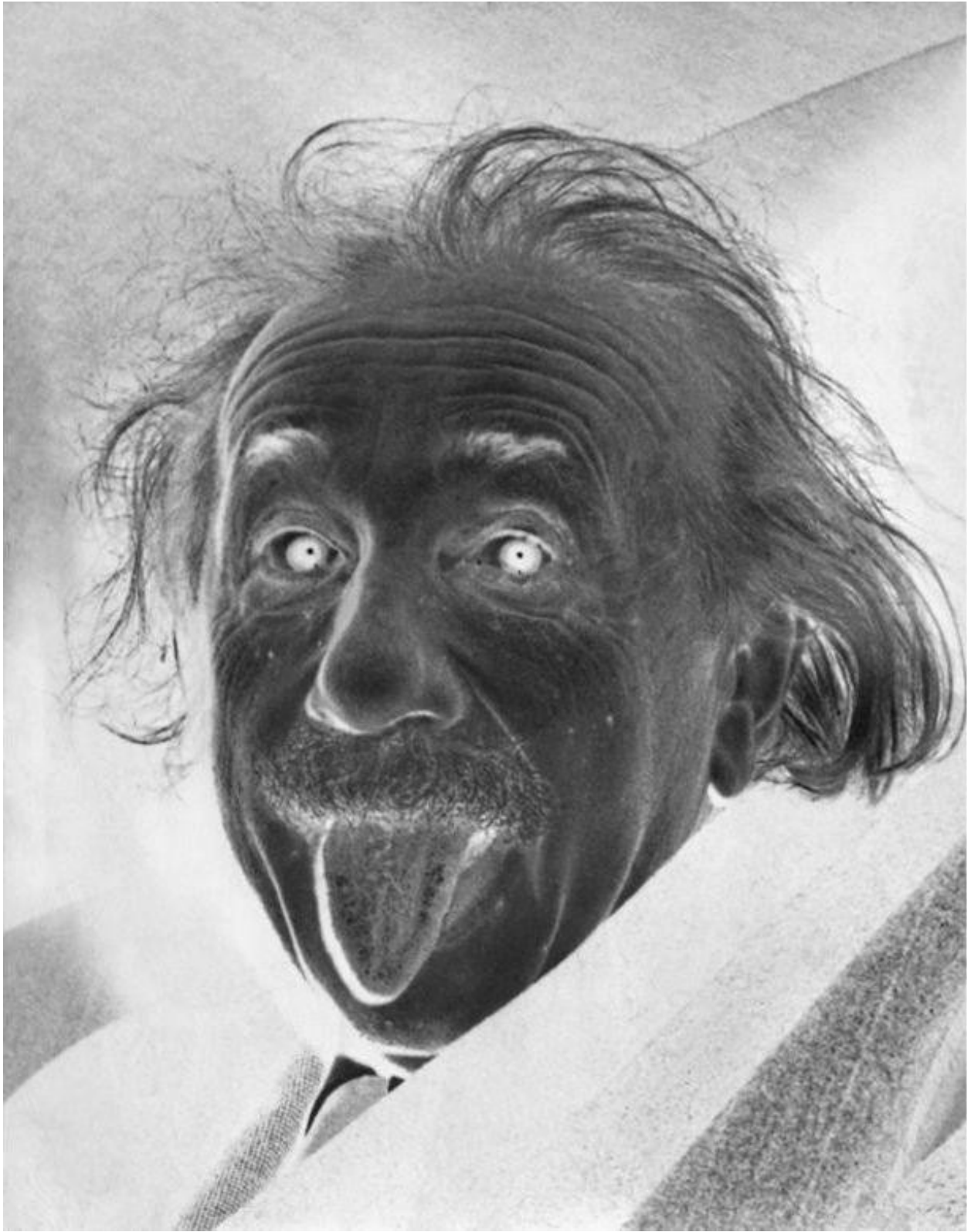
tuple (r, g, b) where r , g , and b are the red, green, and blue values of the pixel respectively. To invert a single colour channel x we can use the formula:

$$\forall x, y, C \in \{R, G, B\}, C_{xy}' = 255 - C_{xy}$$

Use this to implement the function `invert` that inverts the colors of an image.

```
# TODO: Your code here
def invert(img):
    pass

# Use this to test your solution
invert(einstein).show()
```



Exercise 2: Grayscale Images [] []

Next, we will try to turn a coloured image into grayscale. Here's an example:

|rainbow.jpeg|rainbow_gray|

2.1 To make an image grayscale, we average the red, green, and blue values of each pixel and assign the result to all three channels. We can write this mathematically as:

$$\forall x, y, C \in \{R, G, B\}, C_{xy}' = \frac{R_{xy} + G_{xy} + B_{xy}}{3}$$

Implement a function `grayscale` which takes an image as input and returns a new image that is the grayscale version of the input image

```
# TODO: Your code here
def grayscale(img):
    pass

# Use this code to test your solution
rainbow = SimpleImage("assets/rainbow.jpeg")
rainbow.show()
grayscale_rainbow = grayscale(rainbow)
grayscale_rainbow.show()
```

2.2 Can you make only the left side of the image gray scale like in the image below?

rainbow_half_gray.jpeg

Hint: You only need to change one line in your previous filter.

```
# TODO: Your code here
def half_grayscale(img):
    pass

# Use this code to test your solution
rainbow = SimpleImage("assets/rainbow.jpeg")
rainbow.show()
half_grayscale_rainbow = half_grayscale(rainbow)
half_grayscale_rainbow.show()
```

Exercise 3: Flip Images [] []

Another filter is **flip**. Vertical flip turns an image upside down; horizontal flip reverses the image left to right. Here's an example:

|soccer_ball.jpeg|soccer_ball_flipped|

3.1 Implement a function `flip_horizontal` that flips an image `img` horizontally, just like in this example above.

```
# TODO: Your code here
def flip_horizontal(img):
    pass

# Use this function to test your code
soccerball = SimpleImage("assets/soccer_ball.jpeg")
soccerball.show()
flipped_soccerball = flip_horizontal(soccerball)
flipped_soccerball.show()
```

Exercise 4: Rotating Images [] []

Finally, let's rotate the image. First, we do this counter clockwise. Example:

|square.jpeg|square_left|

4.1 Implement the `rotate_left` function.

```
# TODO: Your code here
def rotate_left(img):
    pass

# Use this to test your code
square = SimpleImage("assets/square.jpeg")
square.show()
rotated_left_square = rotate_left(square)
rotated_left_square.show()
```

4.2 Now implement `rotate_right` function, but use maxially **four lines of code** and no loop!

```
# TODO: Your code here
def rotate_right(img):
    pass

# Use this to test your code
square = SimpleImage("assets/square.jpeg")
square.show()
rotated_right_square = rotate_right(square)
rotated_right_square.show()
```

(Optional) Exercise 5: Make your own Filter [] []

Just for fun, get creative and write a function implementing any filter of your choosing. You can change colors, move pixels, anything you want to do. Show it to one of your TAs once you're done and tell them about what it does.

```
# TODO: Your code here
```

Congratulations 🎉